

OFFICIAL DEPLOYMENT & TECHNICAL MANUAL

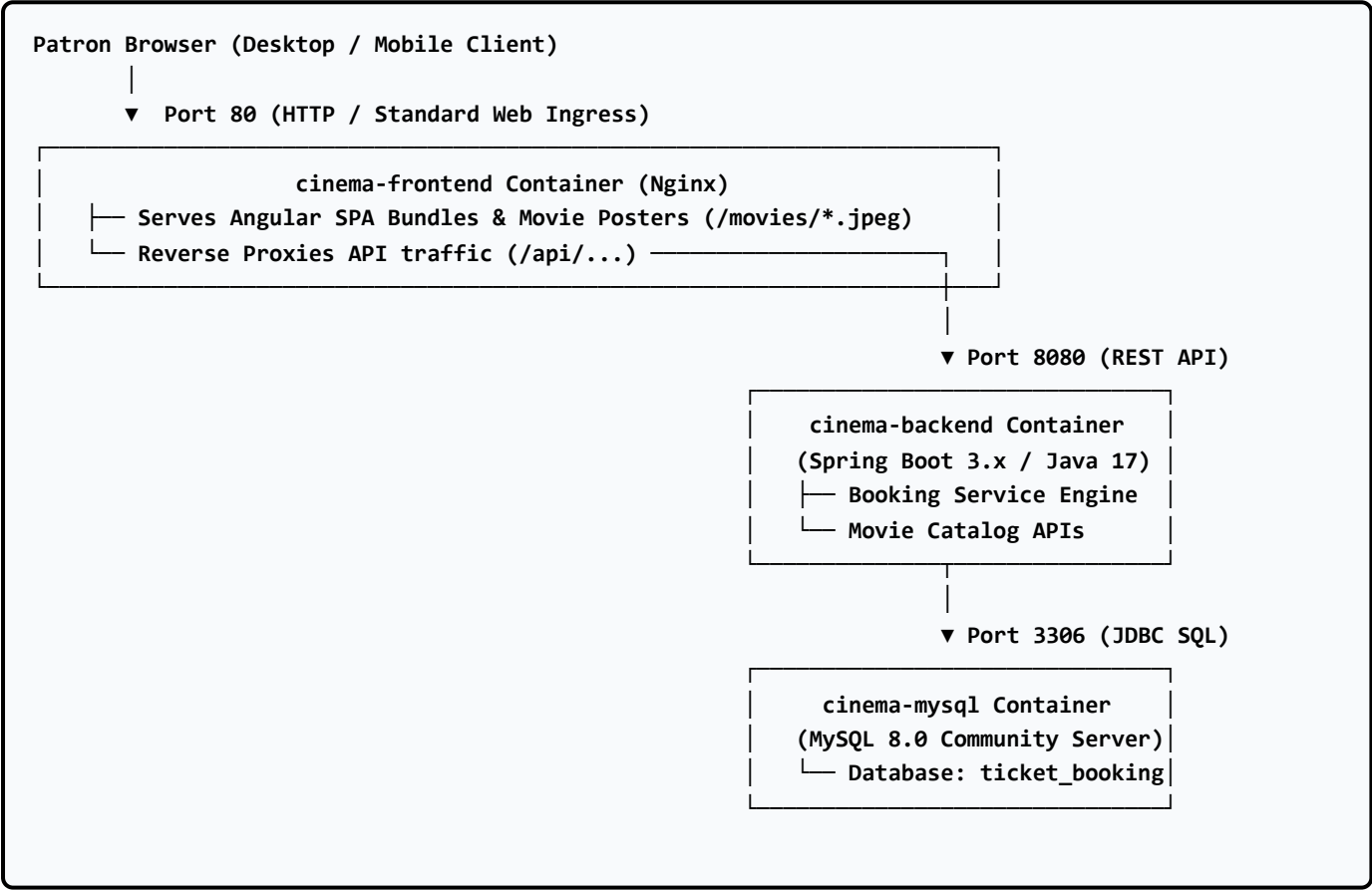


Hari's Theater - Movie Ticket Booking System

Auditorium Address: Sanarapatti, Perumbalai, Pennagaram, Dharmapuri - 636811 | **Infrastructure:** 3 Multiplex Auditoriums • 9 Movie Simultaneous Engine

This technical specification covers the architecture, MySQL database design, tri-screen scheduling rules, and the multi-container Docker cluster deployment for Hari's Theater.

1. System Architecture Diagram



2. Screen Layout & Tri-Session Schedule Matrix

Auditorium	Featured Movies Catalog	Showtime Slots
Screen 1 Dolby Atmos 4K	1. Jailer 2 (₹250) 2. Leo (₹250) 3. Vishwanath & Sons (₹220)	☀ Morning: 10:00–11:00 AM ☀ Matinee: 01:30–02:45 PM 🌙 Night: 06:00–07:15 PM
Screen 2 RGB Laser Cinema	4. The Dark Knight (₹220) 5. Spider-Man Remastered (₹240) 6. Bramayugam (₹200)	☀ Morning: 10:15–11:15 AM ☀ Matinee: 02:00–03:00 PM 🌙 Night: 06:30–07:30 PM
Screen 3 VIP Luxe Cinema	7. Kudumbasthan (₹180) 8. Kalamkaval (₹200) 9. Ekō (₹180)	☀ Morning: 10:00–11:00 AM ☀ Matinee: 01:45–02:30 PM 🌙 Night: 06:15–07:00 PM

3. Complete Database Schema (`ticket\_booking`)

```
-- 1. Database Creation
CREATE DATABASE IF NOT EXISTS ticket_booking;
USE ticket_booking;

-- 2. Movies Catalog Table
CREATE TABLE IF NOT EXISTS movies (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(255) NOT NULL,
  genre VARCHAR(100),
  duration INT,
  price DOUBLE
);

-- 3. Confirmed Passenger Bookings Table
CREATE TABLE IF NOT EXISTS bookings (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  movie_id BIGINT,
  customer_name VARCHAR(255),
  seat_number VARCHAR(10),
  theater VARCHAR(255),
  show_date VARCHAR(50),
  show_time VARCHAR(50),
  total_amount DOUBLE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (movie_id) REFERENCES movies(id)
);

-- 4. Initial Seed Data (All 9 Films with Correct Nomenclature)
INSERT INTO movies (id, title, genre, duration, price) VALUES
(1, 'Jailer 2', 'Action / Crime', 165, 250.0),
(2, 'Leo', 'Action / Thriller', 164, 250.0),
(3, 'Vishwanath & Sons', 'Family Drama', 148, 220.0),
(4, 'The Dark Knight', 'Action / Crime', 152, 220.0),
(5, 'Spider-Man Remastered', 'Action / Sci-Fi', 135, 240.0),
(6, 'Bramayugam', 'Mystery / Horror', 139, 200.0),
(7, 'Kudumbasthan', 'Comedy / Drama', 140, 180.0),
(8, 'Kalamkaval', 'Crime / Investigation', 145, 200.0),
(9, 'Ekō', 'Adventure / Drama', 125, 180.0);
```

4. Seating Matrix Legend

The cinema hall renders a 24-seat layout (Rows A–D, Columns 1–6) with deterministic state feedback:

-  **AVAILABLE**: Seat is open for reservation. Clicking toggles selection.
-  **SELECTED**: Currently picked seat by customer awaiting confirmation.
-  **OCCUPIED**: Locked / reserved seat; disabled from click interaction.

5. Docker Multi-Stage Container Setup

5.1 Backend Multi-Stage Dockerfile (`backend/Dockerfile`)

```
# Stage 1: Build the Executable JAR with Maven
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Minimalist Lightweight Production Runtime
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.2 Nginx Reverse Proxy Configuration (`frontend/nginx.conf`)

```
server {
    listen 80;
    server_name localhost;

    # Serve static Angular SPA bundles & movie posters
    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        try_files $uri$uri/ /index.html;
    }

    # Direct proxying of REST API requests to Spring Boot container
    location /api/ {
        proxy_pass http://backend:8080/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

5.3 Frontend Multi-Stage Dockerfile (`frontend/Dockerfile`)

```
# Stage 1: Build Angular Static Artifacts
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build -- --configuration production

# Stage 2: Serve using Nginx Alpine
FROM nginx:alpine
COPY --from=builder /app/dist/frontend/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```


5.4 Docker Compose Orchestrator (`docker-compose.yml`)

```
version: '3.8'

services:
  # 1. Database Tier
  mysql-db:
    image: mysql:8.0
    container_name: cinema-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: ticket_booking
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
      - ./backend/src/main/resources/data.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - cinema-net

  # 2. Application Server Tier
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: cinema-backend
    restart: on-failure
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql-db:3306/ticket_booking?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: rootpassword
      SPRING_JPA_HIBERNATE_DDL_AUTO: update
    ports:
      - "8080:8080"
    depends_on:
      - mysql-db
    networks:
      - cinema-net

  # 3. Web & Proxy Tier
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: cinema-frontend
    restart: always
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - cinema-net

volumes:
  db_data:
```

```
networks:
  cinema-net:
    driver: bridge
```

6. Step-by-Step Operations & Terminal Runbook

Deployment Stage	Executable PowerShell / Bash Command
1. Build & Start Cluster	<code>docker compose up -d --build</code>
2. Inspect Running State	<code>docker compose ps</code>
3. Stream Combined Logs	<code>docker compose logs -f</code>
4. Stream Backend Logs	<code>docker compose logs -f backend</code>
5. Stop Cluster (Preserve Data)	<code>docker compose stop</code>
6. Re-Launch Stopped Containers	<code>docker compose start</code>
7. Full Teardown	<code>docker compose down</code>
8. Teardown + Reset Database	<code>docker compose down -v</code>

7. Access Points & Verification URLs

- Production Web Application (Nginx Ingress): `http://localhost` (Port 80)
- Backend Direct REST Health / Catalog: `http://localhost:8080/api/movies`
- Database Direct Port: `localhost:3306` (User: `root`, Pass: `rootpassword`)